SYNAPSEOS · THESIS 1

# Revisions After the Proposal Defense

12 October 2026, updated after Dr. Rey's review · Alexandra Sulit, Willard Soriano, Steven Evian Lozano · Page numbers refer to the attached revised proposal (132 pages).

## Response to the Panel

| CONCERN | RESPONSE | WHERE |
|---|---|---|
| **Chapter 1 has no Background of the Study.** | The content was present but had no subsection headings. Chapter 1 is now divided into numbered subsections, beginning with 1.1 Background of the Study, as Chapters 2 and 3 are. | Ch. 1, pp. 1–13 |
| **How is this a computer science study rather than an IT project?** | The core contribution is an algorithm: a static analysis that predicts what a command will change before it runs, so that what it destroys can be backed up and restored. It builds on established program-analysis theory and is evaluated independently of the interface. Chapter 2 now opens with that theory and has a section on command safety, static analysis of shell commands, and recovery systems, and Chapter 3's framework figure puts the algorithm at the centre. | pp. 16, 24, 66, 83 |
| **The objectives are not aligned with Computer Science; objectives 4–5 were system design and HCI.** (Sir Joel; Dr. Rey) | Adopted Dr. Rey's five objectives, all algorithmic: an effect-semantics model, an effect-composition algorithm, a minimum-cost recovery plan, evaluation against pattern-based and fail-closed baselines (on data loss, verified restoration, and needless confirmation), and ablation. Each has its own research question, and Chapter 3 is reorganized to follow them: the model, algorithm, and plan in §2.1(a)–(c), the evaluation in §3.1, and the ablation in §3.2. SynapseOS is now the experimental platform, and the user study validates the algorithm in use. New title: *Recoverability Analysis of Language-Model-Generated Shell Commands, Deployed in the SynapseOS Conversational Interface*. | §1.3–1.4, pp. 4–6; Ch. 3, pp. 48–104 |
| **How much of the result comes from the algorithm, and how much from its rule tables?** (adviser, earlier review) | Now measured for both of the algorithm's parts. With the same rule table, backups fall from **89.5%** to **35.4%** without composition, **45.5%** without runtime-target resolution, and **32.0%** with neither (each $p < 10^{-42}$). Data loss without warning stays 0% in every case. The backups come from the algorithm's two parts, not the table; the 0% comes from the table and its default of asking about anything it does not know. | §3.2, pp. 93–95 |

## Limitations We Disclose

- Undo currently reverts the most recent change regardless of folder; it will be scoped before data collection (p. 103).
- The small local language model sometimes mistranslates loosely worded requests, as already noted in the scope (p. 12); this does not affect the algorithm.
- Testing after the evaluation found seven defects in the analyzer, all fixed; none was triggered by any test command, and re-running every test changed no result (p. 88).

The results at a glance are on page 2. Supporting material (the comparison with open-source tools, a recorded end-to-end verification, and per-command results for all 777 test commands) is available on request.

# Results at a Glance

The algorithm was tested on 777 shell commands. Each command's true effect was found by running it in a sealed sandbox and comparing the files before and after, so no command was labeled by hand. Each checker was scored on three things (§1.2(a)–(b) and §3.1, pp. 57–58 and 83–93).

## How the Algorithm Compares

| CHECKER | DESTRUCTIVE COMMANDS RUN WITHOUT WARNING (lower is better) | DESTROYED FILES BACKED UP, UNDO VERIFIED (higher is better) | HARMLESS COMMANDS ASKED ABOUT (lower is better) |
|---|---|---|---|
| **Pattern list** a baseline in the style of today's checkers | 41% | 42%* | 8% |
| **Strict list** asks about anything it does not recognize | 0% | 0% | 27% |
| **Codex CLI's check** open-source AI coding tool | 97%† | — | 0.3% |
| **cc-safety-net, strictest setting** open-source guard for coding agents | 73%† | — | 4% |
| **The recoverability analysis** | **0%** | **89.5%** | **9%** |

* The pattern list's backups were not verified by undo. † Of the 325 commands that destroy files; the other rows are of all 405 destructive commands. The coding tools make no backups.

## What Switching Parts of the Algorithm Off Shows

| THE ALGORITHM… | RUN WITHOUT WARNING | BACKED UP, UNDO VERIFIED | HARMLESS ASKED ABOUT |
|---|---|---|---|
| **in full** | 0% | **89.5%** | 9% |
| **without composition** seeing through `find -exec`, `xargs`, `sudo`, loops | 0% | 35.4% | 66% |
| **without resolution** the safe trial runs that find which files a command touches | 0% | 45.5% | 19% |
| **without both** its list of known commands alone | 0% | 32.0% | 67% |

§3.2, Table 3.5, pp. 93–95. Every drop is significant (each $p < 10^{-42}$).

## What It Means

- **As safe as the strictest list:** no destructive command ran without warning (0 of 405).
- **The only checker that can undo:** it backed up 89.5% of destroyed files first, and of 631 commands run for real and then undone, 630 were restored exactly (p. 90).
- **It asks a third as often** as the strict list (9% against 27%).
- **The algorithm drives the result, not its command list:** without its two parts, backups fall to 32%, and each part needs the other. Safety does not depend on them; usefulness does.
- **Existing tools are not built for personal files:** the coding tools' checks rely on git to undo mistakes, and let 73–97% of file-destroying commands through (p. 89).
- **It works end to end:** run through the full system on real files, undo restored deleted files only with the algorithm enabled (p. 79).